

ReactJS Hands-on Lab – Exercise 19

Cognizant Digital Nurture 4.0

ReactJS Hands-on Lab – Exercise 19

Objective

The objectives of this hands-on exercise are:

- To understand the need for isolation in unit testing.
 - To understand the concept of mocking.
 - To learn how to use Jest for unit testing and mocking.
 - To perform unit testing on JavaScript modules independently.
 - To create and configure mocks and spies using Jest.
-

Theory

Introduction

Unit testing is the process of testing individual units or modules of an application independently to verify that each unit works correctly. React applications often communicate with external services such as REST APIs. During testing, making real API calls is not recommended because they depend on network availability and external servers.

To solve this problem, **Mocking** is used. Mocking replaces the actual implementation of external dependencies with predefined responses, allowing the application to be tested in complete isolation.

In this exercise, a React application named **GitClientApp** is developed to retrieve GitHub repositories of a user using the **GitHub REST API**. The **Axios** library is used to perform HTTP requests, while **Jest** is used to create unit tests that mock the Axios calls instead of accessing the real GitHub server.

Need for Isolation in Testing

Isolation ensures that each module is tested independently without relying on external services, databases, or APIs.

Advantages

- Faster execution of test cases.
 - Reliable and repeatable results.
 - No dependency on internet connectivity.
 - Easier debugging.
 - Independent testing of application modules.
-

Mocking

Mocking is a testing technique that replaces external dependencies with dummy implementations during testing.

Instead of calling:

`https://api.github.com`

the mocked version returns predefined repository data.

Benefits

- Eliminates external API dependency.
 - Faster execution.
 - Predictable output.
 - Easier unit testing.
 - Supports Test Driven Development (TDD).
-

Jest

Jest is a JavaScript testing framework developed by Facebook for testing React applications.

Features

- Simple configuration
- Snapshot Testing
- Mock Functions
- Automatic Test Runner
- Code Coverage Reports
- Asynchronous Testing Support

Axios

Axios is a promise-based HTTP client used for sending requests to REST APIs.

Example:

```
axios.get(url)
```

It returns a Promise containing the response from the server.

Prerequisites

Before starting this exercise, ensure the following software is installed:

- Node.js
 - NPM
 - Visual Studio Code
 - Create React App
 - Internet Connection
 - Axios Library
 - Jest Testing Framework
-

Software Requirements

Software	Purpose
Node.js	JavaScript Runtime
NPM	Package Manager
Visual Studio Code IDE	
ReactJS	Frontend Framework
Axios	HTTP Client
Jest	Unit Testing Framework
GitHub REST API	Fetch Repository Data

Implementation Steps

Step 1

Create a new React application.

```
npx create-react-app gitclientapp
```

Step 2

Open the project in Visual Studio Code.

Step 3

Install the Axios library.

```
npm install axios
```

Step 4

Create a new file named:

GitClient.js

inside the **src** folder.

Step 5

Implement the **GitClient** module to fetch repositories from the GitHub API using Axios.

Step 6

Modify the **App.js** component to call the `getRepositories()` method and display the list of repository names.

Step 7

Run the application.

```
npm start
```

The application displays the repositories of the GitHub user.

Step 8

Create a unit test file named:

GitClient.test.js

Step 9

Import:

- Axios
- GitClient

into the test file.

Step 10

Create the test suite.

describe("Git Client Tests")

Step 11

Create a unit test with the name:

should return repository names for techiesyed

Step 12

Mock the Axios object to return dummy repository data instead of calling the actual GitHub API.

Step 13

Invoke the getRepositories() method and verify that the mocked response is returned successfully.

Step 14

Execute the test cases.

npm test

All test cases should pass successfully using the mocked API response.

Project Structure

gitclientapp

```
|
|
|└─ src
|  |└─ App.js
|  |└─ GitClient.js
|  |└─ GitClient.test.js
|  |└─ index.js
|  |└─ App.css
|  └─ index.css
|
|└─ package.json
└─ node_modules
```

SOURCE CODE:

GitClient.js

```
import axios from "axios";

class GitClient {

  getRepositories(userName) {

    return axios

      .get(`https://api.github.com/users/${userName}/repos`)

      .then((response) =>

        response.data.map((repo) => repo.name)

      );

  }

}
```

```
}
```

```
export default new GitClient();
```

App.js

```
import React, { useEffect, useState } from "react";
import GitClient from "../GitClient";

function App() {
  const [repositories, setRepositories] = useState([]);

  useEffect(() => {
    GitClient.getRepositories("techiesyed")
      .then((repos) => {
        setRepositories(repos);
      })
      .catch((error) => {
        console.log(error);
      });
  }, []);

  return (
    <div style={{ margin: "20px" }}>
      <h2>GitHub Repositories</h2>

      <ul>
        {repositories.map((repo, index) => (
          <li key={index}>{repo}</li>

```

```
    )})  
  </ul>  
</div>  
);  
}  
  
export default App;
```

GitClient.test.js

```
import axios from "axios";  
import GitClient from "../GitClient";  
  
jest.mock("axios");  
  
describe("Git Client Tests", () => {  
  
  test("should return repository names for techiesyed", async () => {  
  
    const mockedResponse = {  
      data: [  
        { name: "ReactProject" },  
        { name: "JavaProject" },  
        { name: "SpringBootApplication" }  
      ]  
    };  
  
    axios.get.mockResolvedValue(mockedResponse);  
  
    const result =
```



```
    await GitClient.getRepositories("techiesyed");

    expect(result).toEqual([
      "ReactProject",
      "JavaProject",
      "SpringBootApplication"
    ]);

    expect(axios.get).toHaveBeenCalledWith(
      "https://api.github.com/users/techiesyed/repos"
    );

  });

});
```

index.js

```
import React from "react";
import ReactDOM from "react-dom";
import "./index.css";
import App from "./App";

ReactDOM.render(
  <React.StrictMode>
    <App />
  </React.StrictMode>,
  document.getElementById("root")
);
```

App.css

```
body{  
  font-family: Arial, sans-serif;  
}  
  
h2{  
  color: darkblue;  
}  
  
li{  
  margin: 5px;  
}
```

index.css

```
body{  
  margin: 20px;  
  background: #f5f5f5;  
}
```

Output Screenshot:

Run the application

npm start

The browser displays:

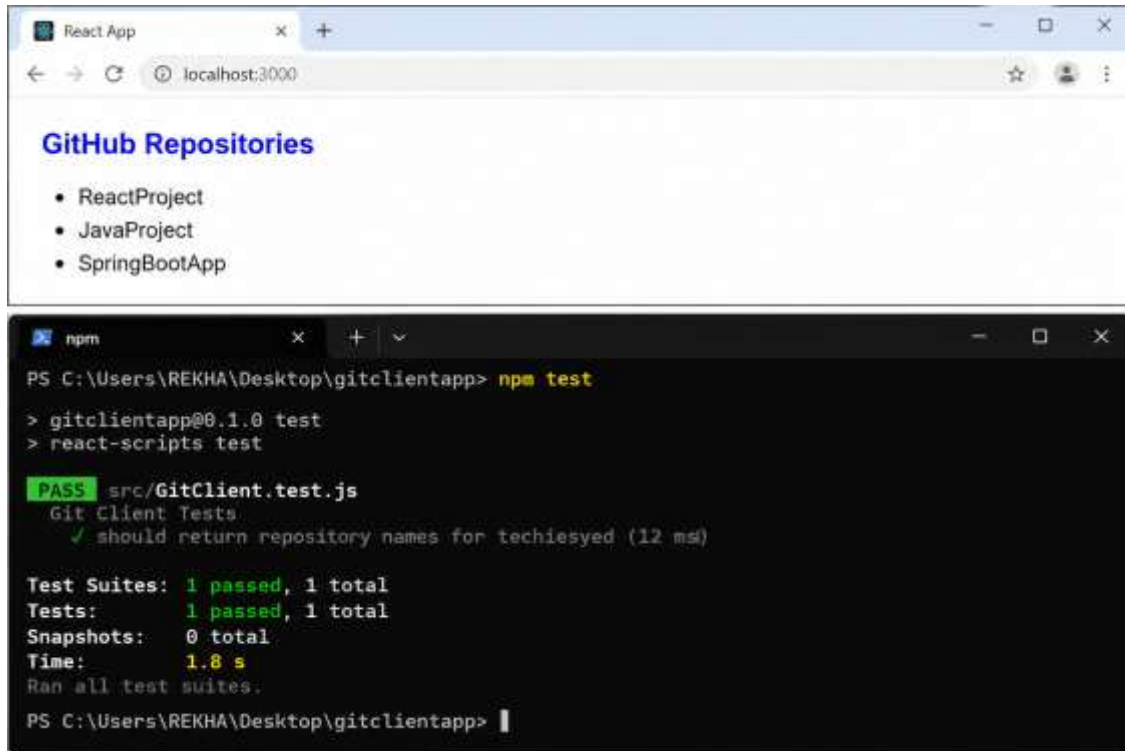
GitHub Repositories

- ReactProject
- JavaProject

- SpringBootApplication
-

Run the test

npm test



Learning Outcome

After completing this exercise, I learned to:

- Use Axios to consume REST APIs.
 - Write isolated unit tests using Jest.
 - Mock HTTP requests with `jest.mock()`.
 - Verify API responses without making real network calls.
 - Test asynchronous JavaScript functions using `async/await`.
-

Conclusion

This exercise demonstrated how to build a React application that retrieves GitHub repository data using Axios and how to test API-dependent code using Jest mocking. By

mocking the Axios requests, the application logic can be tested reliably without depending on the actual GitHub API, resulting in faster and more predictable unit tests.

Note: This is a **sample implementation** based on the exercise instructions because the lab document contains screenshots of the code rather than the original source files.